



www.mech-lang.org

A Rust-Native Embeddable Reactive Numerical Language for Heterogeneous Computing in Robotics

Corey Montella, Aung Kant Ko, Steven McPhillimey

Motivation

- Rust’s memory safety and systems-level control are useful in robotics. Numerical kernels can still become lengthy and hard to check against the original equations, especially when each backend requires specialized code.
- Mech complements Rust by letting programmers express kernels as typed matrix equations. Its compiler handles backend specialization, and its reactive runtime supports successive sensor updates. We examine these features using a bearing-only extended Kalman filter (EKF).

Embeddable

Rust applications can link Mech’s runtime to load source or bytecode, or use a compiled numerical kernel. The interface below compiles the EKF, binds live measurements and exposes persistent state by name. Rust functions and input drivers connect it to the surrounding application.

AOT kernel from Rust

```
usemech::kernel::{Backend,Kernel};
letsource=include_str!("ekf.mec");
letkernel=Kernel::from_source(source)
.input("bearing",[-0.55;4])
.export("state")
.compile(Backend::AotSimd)?;
letmutekf=kernel.start()?;
ekf.turn([("bearing",[-0.54;4])]);
letstate=ekf.state("state");
```

Feature kernel-aot. compile emits the dylib, start creates a session, and turn applies one checked update to four filters. state borrows the accepted result. ekf.mec uses the f32 benchmark equations.

Checked dylibs

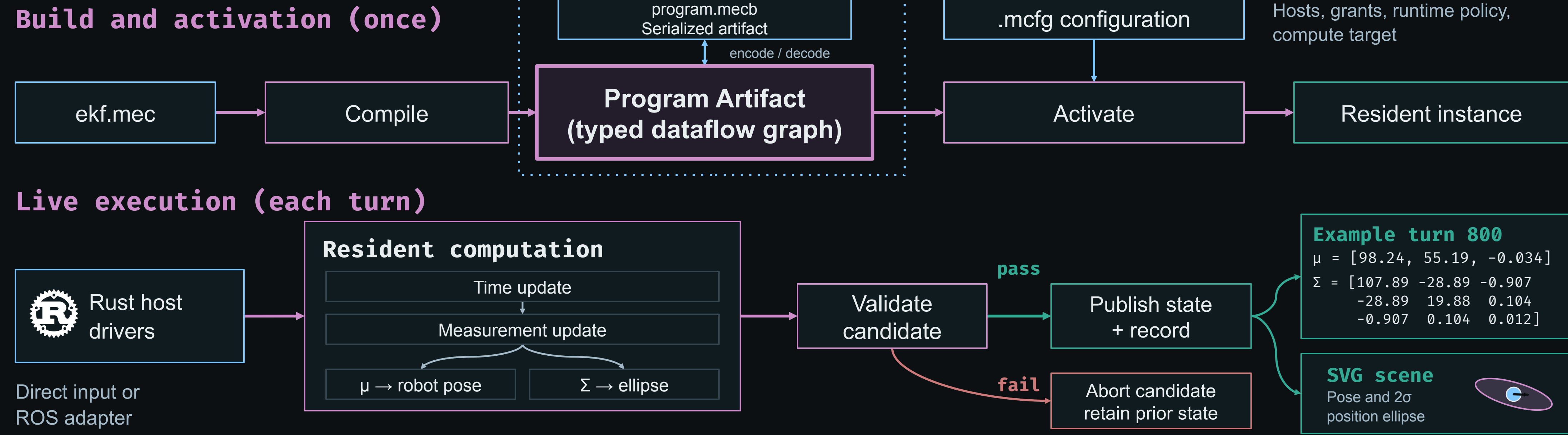
Artifact	Bytes	M/s ± MAD	Peak RSS (MiB)
Mech scalar	33,544	14.671 ± 0.008	2.69 ± 0.00
Rust scalar	50,016	21.121 ± 0.005	2.69 ± 0.00
Mech SIMD-4	33,864	34.863 ± 0.010	2.69 ± 0.00

Mech’s smaller files primarily reflect linker and segment layout. Rust’s specialized scalar loop is faster than scalar Mech. Mech SIMD-4 packs four filters per vector and reaches higher throughput.

1 thread, 10,000 filters × 200 turns, n=7. Median ± MAD. RSS covers the whole loader process. Compile/load/warmup excluded.

Reactive

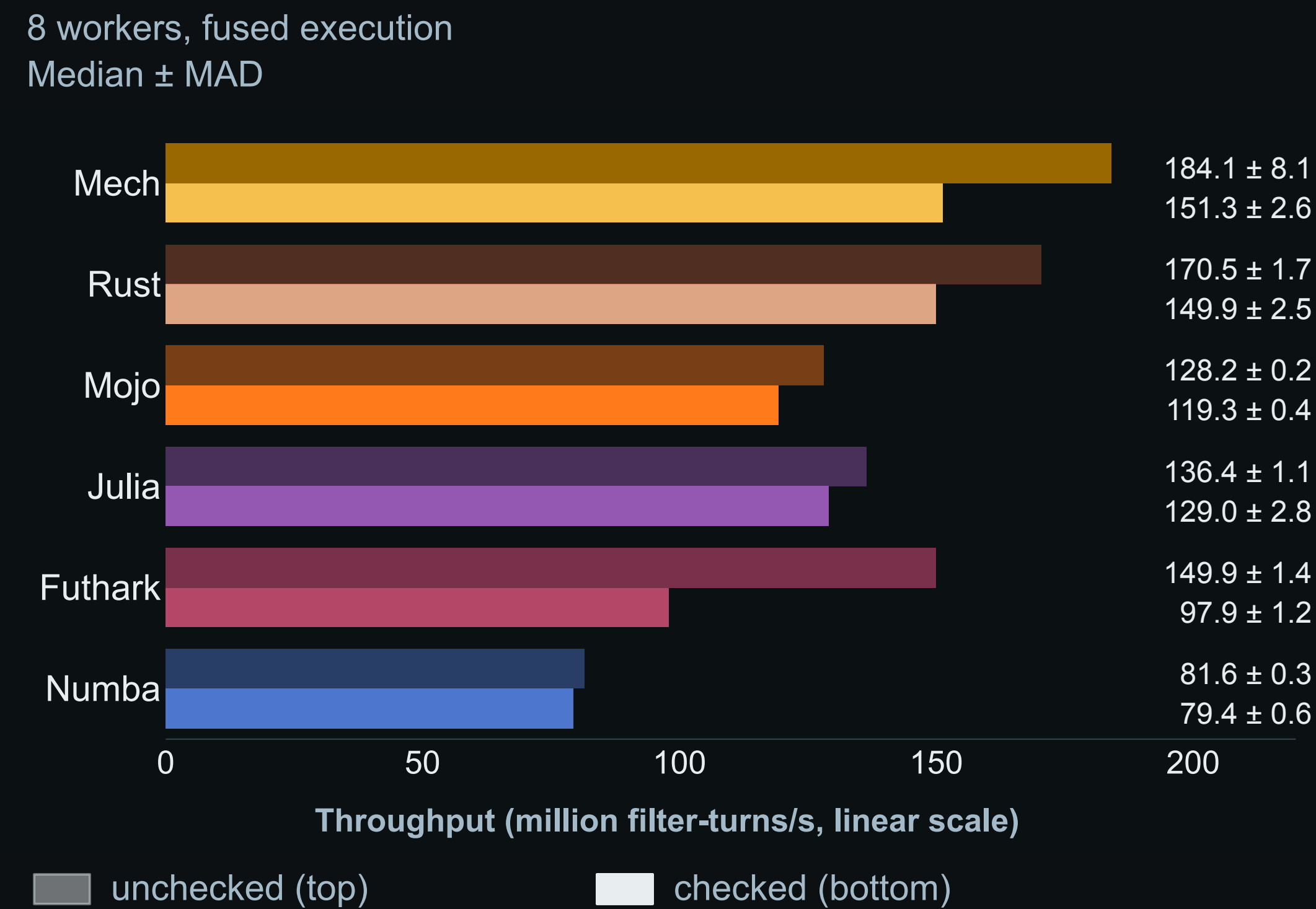
Robotics applications must incorporate new sensor data while retaining estimator state. Rust host drivers submit updates to the resident Mech graph, directly or through ROS adapters. A turn computes candidate state and validates it before publication. Rejection retains the previous state.



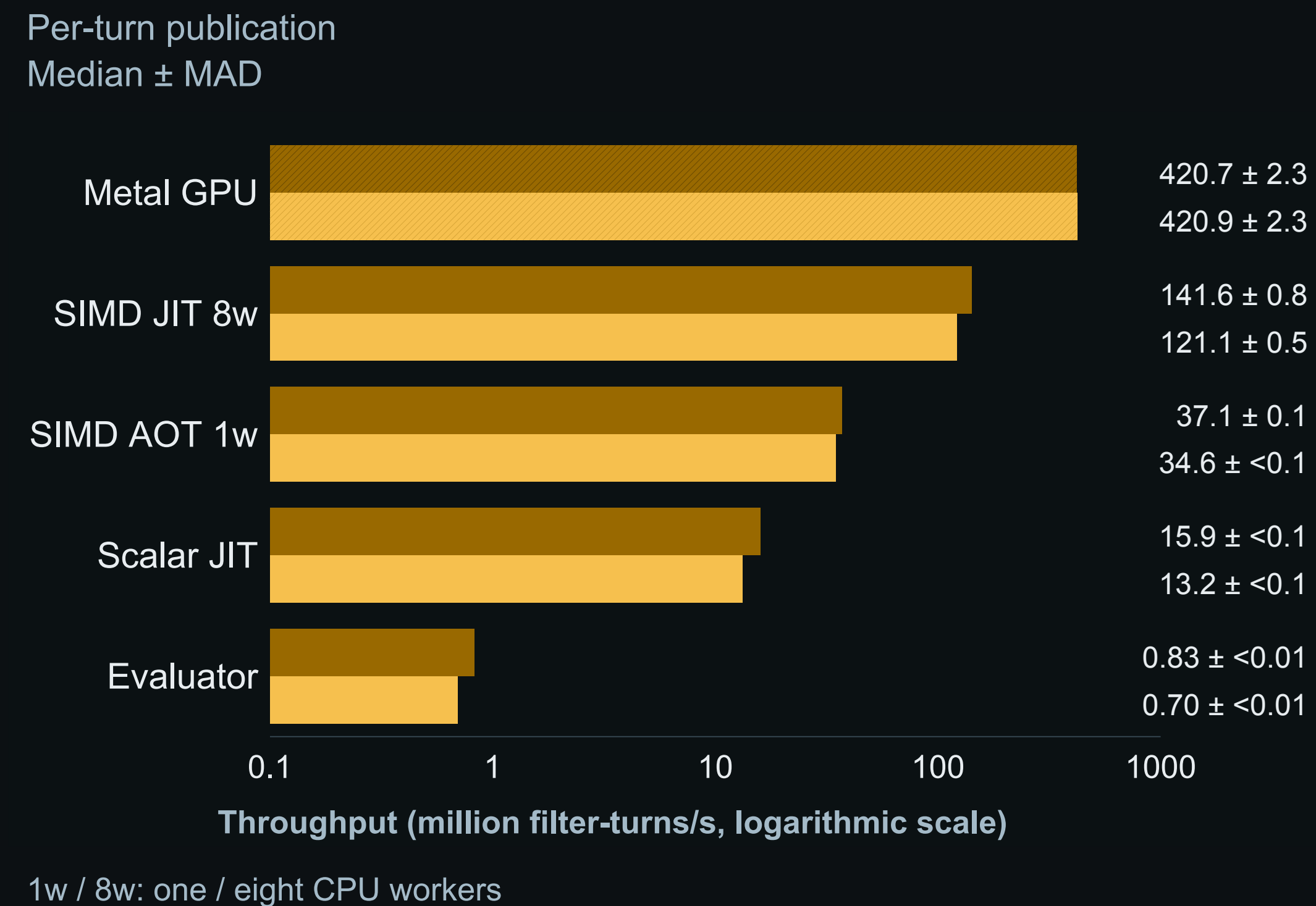
Heterogeneous

All Mech backend measurements use one f32 source version of the EKF shown at right. SIMD JIT and AOT share the numerical graph and Cranelift code generator. JIT compiles on demand. AOT emits a native library before deployment, allowing the host to omit the compiler.

Mech and Rust show similar throughput

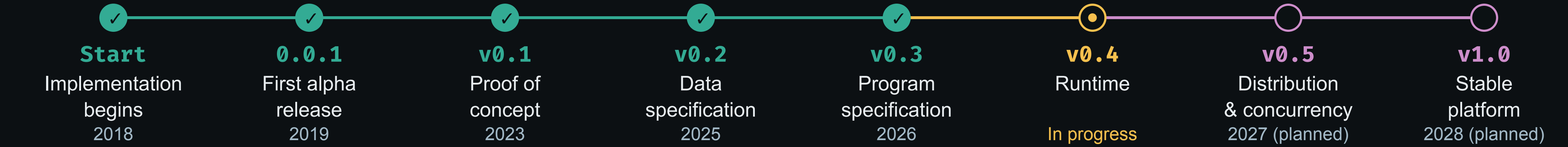


Backend choice changes EKF throughput



Status and Future Work

Mech is in beta. The EKF results show that shorter numerical code can achieve performance comparable to specialized Rust. Stable embedding APIs and tests across more numerical workloads are needed as the runtime and backend support develop.



Experience this poster as a live reactive program. Mech’s WebAssembly runtime supports the CPU and WGPU comparison online.

Benchmark setup Apple M1: 4P + 4E CPU, 8-core GPU, 8 GB, macOS 15.6.1. Charts: f32, 500,000 filters × 40 turns, n=10. Median ± MAD.

Rust 1.96.0-nightly ec818fda3 / LLVM 22.1.0, Cranelift 0.131.3. Julia 1.12.6. Mojo 1.1.0 / MAX 26.6.0. Futhark 0.27.1 / ISPC 1.31.0 / LLVM 22.1.8. Python 3.12.14 / NumPy 2.2.6 / Numba 0.61.2. Apple clang 17.0.0.

Mech v0.4 development (package 0.3.6). Backend pairs f4b69052c, CPU eedc1c75b, dylibs dab98af1d. iros-workshop-benchmarks, benchmarks/iros-2026.

[1] Thrun, Burgard & Fox. Probabilistic Robotics. MIT Press, 2005. §7.4.2, Table 7.2 (adapted).

Numerical

Mech offers a concrete syntax that expresses numerical algorithms as typed matrix equations. The EKF below is comparable in scale to the textbook pseudocode [1], retaining its matrix-level notation.

SIMD Rust uses almost five times as many normalized source characters here, mainly for SIMD and batch support. Reusing the Mech source across backends reduces the specialized implementations to maintain.

Compilation produces a typed dataflow artifact. Matrix operations use nalgebra in the ordinary backend; specialized backends compile the graph directly.

Extended Kalman Filter

$$\begin{aligned} \Delta t &:= 0.1 \\ \mathbf{m} &:= [140 \ 12]' \\ \mathbf{Q} &:= \begin{bmatrix} 0.01 & 0 \\ 0 & 0.0025 \end{bmatrix} \\ \mathbf{R} &:= 0.25 \\ \mathbf{I} &:= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \\ \tilde{\boldsymbol{\mu}} &:= [55 \ 25 \ 0.4]' \\ \tilde{\boldsymbol{\Sigma}} &:= \begin{bmatrix} 100 & 0 & 0 \\ 0 & 100 & 0 \\ 0 & 0 & 0.15 \end{bmatrix} \end{aligned}$$

Time Update

$$\begin{aligned} \boldsymbol{\theta} &:= \boldsymbol{\mu}[3] \\ \mathbf{d} &:= \mathbf{v} * \Delta t \\ \mathbf{G} &:= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & d * \cos(\boldsymbol{\theta}) \\ 0 & 0 & 1 \end{bmatrix} \\ \mathbf{V} &:= \begin{bmatrix} \cos(\boldsymbol{\theta}) * \Delta t & 0 \\ \sin(\boldsymbol{\theta}) * \Delta t & 0 \\ 0 & \Delta t \end{bmatrix} \\ \bar{\boldsymbol{\mu}} &:= \boldsymbol{\mu} + [\mathbf{d} * \cos(\boldsymbol{\theta}) \\ \mathbf{d} * \sin(\boldsymbol{\theta}) \\ \omega * \Delta t] \\ \bar{\boldsymbol{\Sigma}} &:= \mathbf{G} ** \tilde{\boldsymbol{\Sigma}} ** \mathbf{G}' + \mathbf{V} ** \mathbf{Q} ** \mathbf{V}' \end{aligned}$$

Measurement Update

$$\begin{aligned} \delta &:= \mathbf{m} - \bar{\boldsymbol{\mu}}[1..=2] \\ \delta x &:= \delta[1] \\ \delta y &:= \delta[2] \\ q &:= \delta x^2 + \delta y^2 \\ \hat{z} &:= \text{atan2}(\delta y, \delta x) - \bar{\boldsymbol{\mu}}[3] \\ \mathbf{r} &:= z - \hat{z} \\ \mathbf{v} &:= \text{atan2}(\sin(\mathbf{r}), \cos(\mathbf{r})) \\ \mathbf{H} &:= [\delta y / q, (0 - \delta x) / q, -1] \\ \mathbf{S} &:= \mathbf{H} ** \bar{\boldsymbol{\Sigma}} ** \mathbf{H}' + \mathbf{R} \\ \mathbf{K} &:= (\bar{\boldsymbol{\Sigma}} ** \mathbf{H}') / \mathbf{S}[1] \\ \mathbf{A} &:= \mathbf{I} - \mathbf{K} ** \mathbf{H} \\ \boldsymbol{\mu}_+ &:= \bar{\boldsymbol{\mu}} + \mathbf{K} * \mathbf{v} \\ \boldsymbol{\Sigma}_+ &:= \mathbf{A} ** \bar{\boldsymbol{\Sigma}} ** \mathbf{A}' + (\mathbf{K} ** \mathbf{K}') * \mathbf{R} \\ \boldsymbol{\mu} &= \boldsymbol{\mu}_+ \\ \boldsymbol{\Sigma} &= \boldsymbol{\Sigma}_+ \end{aligned}$$

Normalized source characters

Application source	Mech	Rust textbook	Rust SIMD
Setup	340	486	561
EKF + candidate update	356	1,024	1,119
Validation	383	263	396
Matrix helpers	0	251	310
SIMD + batch support	0	331	2,857
Total	1,079	2,355	5,243

SIMD Rust uses almost five times as many normalized source characters as Mech in this example, mainly for SIMD and batch support.

The time and measurement updates remain explicit matrix equations. Backend code handles vector layout and batching, so the same numerical source can serve scalar and SIMD execution.

Rust’s selected SIMD implementation includes vector adapters and a batch dispatcher with rollback. Mech delegates these operations to reusable compiler and runtime components, which explains its zero entries for helpers and SIMD support.

Programmer-chosen names count as one character per occurrence. Comments and nonliteral whitespace are removed. General library, compiler and runtime internals are excluded.

